



UNIDADE IV

Programação
Estruturada em C

Prof. Me. Fábio Assis

Funções em C

- Função é um bloco de código que executa uma tarefa específica e pode ser reutilizado várias vezes no programa. O uso de funções facilita a organização, legibilidade e manutenção do código.

Estrutura de uma função: `tipo_retorno nome_funcao(Lista de parâmetros) {`
 ...
}

Exemplo:

```
int calcula_quadrado(int num) {  
    return num * num;  
}
```



Usado para retornar
o valor da função.

Em C, apenas um valor
pode ser retornado.

Funções em C

- As funções são utilizadas para dividir um programa em módulos menores e reutilizáveis, facilitando a organização e manutenção do código.

Componentes básicos de uma função:

- Tipo de retorno: define o tipo de dado que a função devolve (int, float, void etc.).
 - Nome da função: identificador usado para chamá-la.
 - Parâmetros: são os dados recebidos (opcional).
 - Corpo da função: instruções que serão executadas.
 - return: interrompe a função e retorna o resultado (exceto em funções do tipo void, pois este tipo não possui retorno).
-
- Ao definir uma função, as variáveis criadas dentro dela são chamadas de variáveis locais, ou seja, só podem ser usadas dentro daquele bloco de função.

Funções em C

Uso comum de funções:

- Usadas para modularizar o código;
- Uma função executa uma operação e retorna um valor.

Procedimento:

- Um procedimento é uma função que não retorna um valor;
- Usamos void para indicar isso.

Exemplo: `void mostra_quadrado(int num) {
 printf("%d\n", num * num);
}`

Funções em C

- Função main: A função main em C deve ser declarada como `int main()`, pois é recomendada e padrão pela linguagem C, conforme o padrão ISO C (normas ANSI/ISO).

```
int main() {  
    ...  
  
    return 0;  
}
```

- A função main é chamada automaticamente quando o programa é iniciado.

Por convenção, a função `main()` retorna um valor inteiro ao sistema operacional ao final da execução:

- Retorna 0 quando o programa termina como esperado;
- Retorna outro valor em caso de erro.

Funções em C

- Chamada de Funções: Para que uma função seja executada, é necessário chamá-la em algum ponto do programa. A chamada é feita usando o nome da função seguido dos argumentos entre parênteses, caso existam. Esses argumentos fornecem os dados que a função precisa para realizar sua tarefa.
- Exemplo: Programa que calcula o quadrado de um número usando função antes da função main().

```
#include<stdio.h>
```

```
int calcula_quadrado(int num) {  
    return num * num;  
}
```

```
int main() {
```

```
    int n1;  
    scanf("%d", &n1);
```

```
    int quad = calcula_quadrado(n1);
```

```
    printf("%d\n", quad);
```

```
    return 0;
```

```
}
```

Funções em C

- Exemplo: Programa que calcula o quadrado de um número usando função com protótipo antes da função main().
- A declaração `int calcula_quadrado(int num);` diz ao compilador que existe uma função que retorna int e recebe um int como parâmetro.

Protótipo ou assinatura da função



```
#include<stdio.h>

int calcula_quadrado(int num);

int main() {
    int n1;
    scanf("%d", &n1);
    int quad = calcula_quadrado(n1);
    printf("%d\n", quad);
    return 0;
}

int calcula_quadrado(int num) {
    return num * num;
}
```

Interatividade

Analise o código abaixo escrito em C e as afirmações a seguir:

- I. A função soma recebe dois inteiros e retorna um inteiro.
- II. A assinatura informa ao compilador que a função vem depois do main.
- III. O programa imprime o resultado da soma de 3 e 5, que é 8.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>
```

```
int soma(int a, int b);
```

```
int main() {  
    printf("Soma: %d\n", soma(3, 5));  
    return 0;  
}
```

```
int soma(int a, int b) {  
    return a + b;  
}
```


Resposta

Analise o código abaixo escrito em C e as afirmações a seguir:

- I. A função soma recebe dois inteiros e retorna um inteiro.
- II. A assinatura informa ao compilador que a função vem depois do main.
- III. O programa imprime o resultado da soma de 3 e 5, que é 8.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>
```

```
int soma(int a, int b);
```

```
int main() {  
    printf("Soma: %d\n", soma(3, 5));  
    return 0;  
}
```

```
int soma(int a, int b) {  
    return a + b;  
}
```

Funções em C: Passagem de Parâmetros

- Em C, a passagem de parâmetros padrão é por valor, o que significa que uma cópia do valor do argumento é passada para o parâmetro da função.
- Alterações no parâmetro dentro da função não afetam a variável original fora da função (a menos que se use ponteiros para passar por referência).
- Normalmente, uma função recebe valores através de parâmetros, que servem como uma forma de comunicação entre funções.
- Esses parâmetros são tratados como variáveis locais durante a execução da função.
- Qualquer modificação feita no parâmetro dentro da função não afeta a variável original que foi passada como argumento no código de chamada.

Funções em C: Passagem de Parâmetros

- Passagem por referência: Para passar um argumento por referência, precisamos passar o valor do endereço de memória (ponteiro). Portanto, para a passagem por referência, você precisa explicitamente passar o endereço de memória da variável original para a função. Isso é feito utilizando o operador de endereço & antes do nome da variável no momento da chamada da função.
- O parâmetro da função, por sua vez, deve ser declarado como um ponteiro para o tipo da variável original (usando o operador *).
- Dentro da função, você pode então usar o operador * para acessar e modificar o valor contido no endereço de memória apontado pelo ponteiro, efetivamente alterando a variável original.

Funções em C: Passagem de Parâmetros

- Exemplo: Passagem por valor para função.

```
#include <stdio.h>
#include <locale.h>

void modificarValor(int x) {
    printf("Dentro da função, antes da modificação: x = %d\n", x);
    x = 100;
    printf("Dentro da função, depois da modificação: x = %d\n", x);
}

int main() {
    setlocale(LC_ALL, "Portuguese");

    int valor = 10;

    printf("Antes da chamada da função: valor = %d\n", valor);
    modificarValor(valor);
    printf("Depois da chamada da função: valor = %d\n", valor);
    return 0;
}
```

Funções em C: Passagem de Parâmetros

Saída na tela:

```
Antes da chamada da função: valor = 10
Dentro da função, antes da modificação: x = 10
Dentro da função, depois da modificação: x = 100
Depois da chamada da função: valor = 10
```

```
#include <stdio.h>
#include <locale.h>

void modificarValor(int x) {
    printf("Dentro da função, antes da modificação: x = %d\n", x);
    x = 100;
    printf("Dentro da função, depois da modificação: x = %d\n", x);
}

int main() {
    setlocale(LC_ALL, "Portuguese");

    int valor = 10;

    printf("Antes da chamada da função: valor = %d\n", valor);
    modificarValor(valor);
    printf("Depois da chamada da função: valor = %d\n", valor);
    return 0;
}
```

Observação:

- Esse exemplo demonstra que em C, quando passamos variáveis por valor (e não por referência), qualquer alteração dentro da função não afeta a variável original no main.

Funções em C: Passagem de Parâmetros

- Exemplo: Este exemplo ilustra a passagem por referência para função (usando ponteiros).

```
#include <stdio.h>
#include <locale.h>

void modifReferencia(int *x) {
    printf("Dentro da função, antes da modificação: *ptr = %d\n", *x);
    *x = 100;
    printf("Dentro da função, depois da modificação: *ptr = %d\n", *x);
}

int main() {
    setlocale(LC_ALL, "Portuguese");

    int valor = 10;

    printf("Antes da chamada da função: valor = %d\n", valor);

    //Passa o endereço de memória de valor
    modifReferencia(&valor);

    printf("Depois da chamada da função: valor = %d\n", valor);

    return 0;
}
```

Funções em C: Passagem de Parâmetros

- Exemplo: Este exemplo ilustra a passagem por referência para função (usando ponteiros).

Saída na tela:

```
Antes da chamada da função: valor = 10
Dentro da função, antes da modificação: *ptr = 10
Dentro da função, depois da modificação: *ptr = 100
Depois da chamada da função: valor = 100
```

```
#include <stdio.h>
#include <locale.h>

void modifReferencia(int *x) {
    printf("Dentro da função, antes da modificação: *ptr = %d\n", *x);
    *x = 100;
    printf("Dentro da função, depois da modificação: *ptr = %d\n", *x);
}

int main() {
    setlocale(LC_ALL, "Portuguese");

    int valor = 10;

    printf("Antes da chamada da função: valor = %d\n", valor);

    //Passa o endereço de memória de valor
    modifReferencia(&valor);

    printf("Depois da chamada da função: valor = %d\n", valor);

    return 0;
}
```

Observação:

- Nesse exemplo, ao passar o endereço de memória de valor para a função *modifReferencia*, a função pôde alterar o valor original da variável *valor* na função *main* através do ponteiro **x*.

Interatividade

Analise as afirmativas a seguir com base no código apresentado:

- I. A função dobrar tenta dobrar o valor de num, mas não altera seu valor original no main.
- II. O valor impresso na tela será 10.
- III. A variável x usada na função dobrar é uma cópia de num.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>
```

```
void dobrar(int x) {  
    x = x * 2;  
}
```

```
int main() {  
    int num = 5;  
    dobrar(num);  
    printf("Valor final: %d\n", num);  
    return 0;  
}
```


Resposta

Analise as afirmativas a seguir com base no código apresentado:

- I. A função dobrar tenta dobrar o valor de num, mas não altera seu valor original no main.
- II. O valor impresso na tela será 10.
- III. A variável x usada na função dobrar é uma cópia de num.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>
```

```
void dobrar(int x) {  
    x = x * 2;  
}
```

```
int main() {  
    int num = 5;  
    dobrar(num);  
    printf("Valor final: %d\n", num);  
    return 0;  
}
```

Escopo de Variáveis em C:

- Em C, o escopo de uma variável define a região do código onde essa variável é visível e pode ser acessada. O escopo de uma variável é determinado pelo local onde ela é declarada dentro do programa.

Em C, existem escopos que determinam a visibilidade das variáveis. Os principais incluem:

- Variáveis locais (escopo de bloco ou de função): Acessíveis apenas dentro do bloco de código ou da função onde são declaradas.
- Variáveis globais: Acessíveis a partir de qualquer parte do código.

Escopo de Variáveis em C:

Escopo Local:

- As variáveis declaradas dentro de uma função têm escopo local, o que significa que elas só são acessíveis dentro dessa função. As variáveis declaradas dentro de um bloco de código (delimitado por chaves {}), como em estruturas de decisão if, ou de repetição, como for, têm um escopo ainda mais restrito, existindo apenas dentro desse bloco.

Vantagens:

- Melhor controle sobre o escopo e o uso da variável.
- Evitam conflitos de nomes com outras partes do programa.
- Facilitam o rastreamento de alterações e reduzem efeitos colaterais.

Desvantagens:

- Não podem compartilhar informações diretamente entre funções.

Escopo de Variáveis em C:

Escopo Global:

- A variável global pode ser usada diretamente em qualquer função, sem a necessidade de passar como parâmetro. Embora convenientes, o uso excessivo de variáveis globais pode dificultar o rastreamento de onde os dados são modificados, tornando o código mais propenso a erros e menos modular.

Vantagens:

- Permitem compartilhar dados entre várias funções.
- Permanecem na memória durante toda a execução do programa.

Desvantagens:

- Difícil rastrear alterações em programas grandes.
- Podem causar conflitos de nomes ou erros devido a alterações não intencionais.
- Consomem mais memória, pois permanecem alocadas durante toda a execução.

Escopo de Variáveis em C:

Exemplo:

```
#include<stdio.h>
```

```
double resultado; ←
```

```
void calcula_quadrado(double num) {  
    resultado = num * num;  
}
```

```
double calcula_soma(double n1, double n2) {  
    double r; ←  
    r = n1 + n2;  
    return r;  
}
```

```
int main() {  
    int a = 2, b = 3; ←  
    resultado = calcula_soma(a, b);  
    printf("%.2lf\n", resultado);  
    calcula_quadrado(resultado);  
    printf("%.2lf\n", resultado);  
    calcula_quadrado(resultado);  
    printf("%.2lf\n", resultado);  
  
    return 0;  
}
```

Variável global

A variável resultado pode ser acessada a partir de qualquer função!

Variável local da função calcula_soma

Variáveis locais da função main

Variáveis locais são acessíveis apenas da função onde foram declaradas.

Interatividade

Analise as afirmativas a seguir com base no código apresentado:

- I. A variável x global vale 5.
- II. A função mostrar imprime o valor 5.
- III. A variável local x dentro da função oculta a global.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>

int x = 5; // global

void mostrar() {
    int x = 10; // local
    printf("x local = %d\n", x);
}

int main() {
    mostrar();
    printf("x global = %d\n", x);
    return 0;
}
```

Resposta

Analise as afirmativas a seguir com base no código apresentado:

- I. A variável x global vale 5.
- II. A função mostrar imprime o valor 5.
- III. A variável local x dentro da função oculta a global.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>

int x = 5; // global

void mostrar() {
    int x = 10; // local
    printf("x local = %d\n", x);
}

int main() {
    mostrar();
    printf("x global = %d\n", x);
    return 0;
}
```

Ponteiros em C:

- Ponteiro é uma variável que armazena um endereço de memória de outra variável. Ou seja, ponteiro guarda o endereço (local na memória) de uma variável, em vez de valores. Serve para modificar valores diretamente.

Para declarar um ponteiro, basta incluir um asterisco antes do nome da variável: `int *p;`

- Neste exemplo, *p* é um ponteiro que pode armazenar o endereço de uma variável do tipo int.

Operadores com ponteiros:

- & (e comercial): retorna o endereço de uma variável.
- * (Asterisco): usado para acessar o valor que está sendo apontado.

Ponteiros em C:

- Principais usos de ponteiros em C.
- Eles permitem manipular diretamente a memória:
- Acesso direto à memória.
 - Alocação dinâmica de memória.
 - Passagem por referência em funções.
 - Manipulação de *arrays*.
 - Manipulação de *strings*.
 - Trabalhar com estruturas (*structs*).

```
#include <stdio.h>
#include <locale.h>

int main() {
    setlocale(LC_ALL, "Portuguese");

    int x = 10;
    int *p = &x;

    printf("Valor de x: %d\n", x);
    printf("Endereço de x: %p\n", &x);
    printf("Valor apontado por p: %d\n", *p);

    *p = 200;
    printf("Novo valor de x: %d\n", x);

    return 0;
}
```

Saída na tela:

```
Valor de x: 10
Endereço de x: 0061FF18
Valor apontado por p: 10
Novo valor de x: 200
```

Arquivos em C:

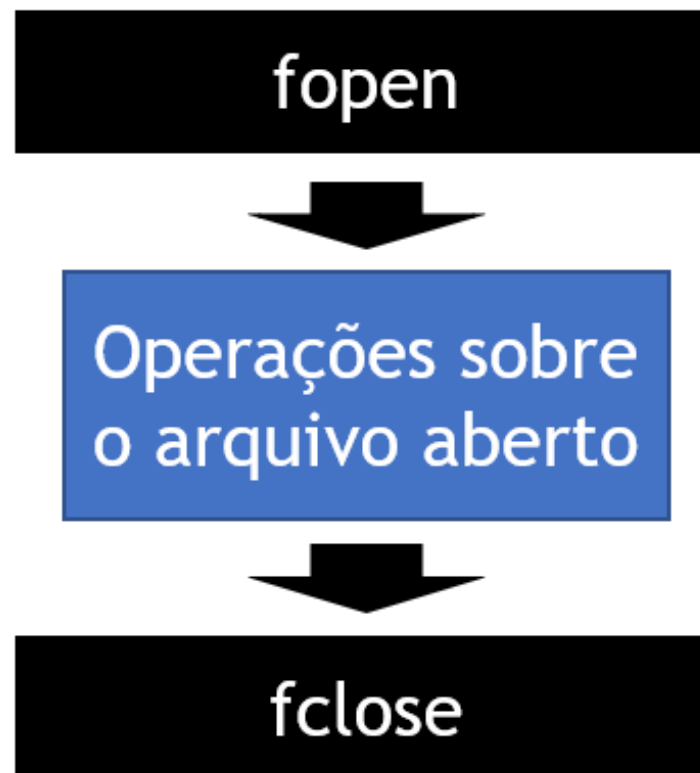
- Arquivos em C são usados para armazenar dados de forma permanente, ou seja, mesmo após o encerramento do programa, as informações permanecem salvas no disco.

O uso de arquivos permite salvar dados entre execuções do programa, possibilitando diversas aplicações práticas, como:

- Armazenamento de dados para uso posterior.
- Gravação de pontuações, progresso de jogos ou status de atividades.
- Geração de relatórios, logs ou estruturas simples de banco de dados.
- Leitura de dados de entrada sem depender do teclado.
- Processamento de arquivos de texto, como .txt, .csv e arquivos binários.

Arquivos em C:

- Para usar arquivos, precisamos primeiro abrir (*fopen*) e depois fechar os arquivos (*fclose*).
- Usamos *fopen*, que recebe o modo de abertura do arquivo.
- Usamos *fclose* para fechar o arquivo.



Arquivos em C:

Principais funções de arquivos (*sdtio.h*)

Função	Descrição
fopen()	Abre (ou cria) um arquivo
fclose()	Fecha o arquivo
fprintf()	Escreve dados formatados no arquivo
fscanf()	Lê dados formatados do arquivo
fgetc()	Lê um caractere do arquivo
fputc()	Escreve um caractere no arquivo
fgets()	Lê uma string do arquivo
fputs()	Escreve uma string no arquivo
feof()	Verifica se chegou ao final do arquivo (EOF)

Arquivos em C:

Exemplo:

```
#include <stdio.h>
```

```
int main() {  
    FILE *f = fopen("dados.txt", "w");  
    fprintf(f, "Olá, arquivo!\n");  
    fclose(f);  
    return 0;  
}
```

Explicação:

- `fopen("dados.txt", "w")` → cria (ou sobrescreve) o arquivo para escrita.
- `fprintf(f, "...")` → escreve no arquivo.
- `fclose(f)` → fecha o arquivo.
- Esse programa criará (ou substituirá) o arquivo `dados.txt` com o texto: Olá, arquivo!

Interatividade

Analise as afirmativas a seguir com base no código apresentado:

- I. O programa cria (ou sobrescreve) um arquivo chamado saida.txt.
- II. A função fprintf escreve no arquivo, não na tela.
- III. O programa não precisa fechar o arquivo com fclose().

```
#include <stdio.h>

int main() {
    FILE *f = fopen("saida.txt", "w");
    fprintf(f, "Exemplo de escrita.\n");
    fclose(f);
    return 0;
}
```

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

Resposta

Analise as afirmativas a seguir com base no código apresentado:

- I. O programa cria (ou sobrescreve) um arquivo chamado saida.txt.
- II. A função fprintf escreve no arquivo, não na tela.
- III. O programa não precisa fechar o arquivo com fclose().

```
#include <stdio.h>

int main() {
    FILE *f = fopen("saida.txt", "w");
    fprintf(f, "Exemplo de escrita.\n");
    fclose(f);
    return 0;
}
```

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

Referências

- BACKES, André. *Linguagem C: completa e descomplicada*. São Paulo: Grupo GEN, 2018.
- CELES, Waldemar; RANGEL, Renato; EIRAS, José. *Introdução a estruturas de dados: com técnicas de programação em C*. Rio de Janeiro: Elsevier/Grupo GEN, 2016.
- DAMAS, Luiz. *Linguagem C*. 10. ed. Rio de Janeiro: LTC, 2011.
- DEITEL, Paul; DEITEL, Harvey. *C: como programar*. 7. ed. São Paulo: Pearson Prentice Hall, 2013.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. São Paulo: Editora Saraiva, 2016.

ATÉ A PRÓXIMA!